@CACHEPLANE/RENDER · ENTERPRISE GUIDE

The Enterprise Guide to Generative UI in Angular

Agents that render UI — without coupling to your frontend

cacheplane.io · 2026

Contents

01 The Coupling Problem

02 Declarative UI Specs & the json-render Standard

03 The Component Registry

04 Streaming JSON Patches

05 State Management & Computed Functions

CHAPTER 1

The Coupling Problem

Chapter 3: The Coupling Problem

The Antipattern You're Already Writing

If you've integrated a generative agent into an Angular application, you've probably written something like this:

This is clean, readable, and completely reasonable for a prototype. It is also quietly catastrophic for a production system at scale.

The problem isn't the switch statement. The problem is what the switch statement represents: a hardcoded, implicit contract between two systems that were never designed to evolve together.

Bidirectional Dependency

When a component switches on `agentOutput.type`, you've created a dependency that runs in both directions. The frontend depends on the agent emitting specific, known type strings. The agent—or more precisely, the engineers tuning it—starts depending on the frontend to remain stable. If you add a `'timeline'` component to your library, that string has to appear in the agent's output schema before the component is useful. If the agent starts emitting `'comparison-table'` to reflect a new capability, the frontend silently falls through to the fallback until a developer manually adds the case.

Neither system can move independently. You've coupled iteration velocity in two entirely separate codebases through an undocumented string literal.

Frontend Engineers Become the Bottleneck

In practice, this coupling has a human cost. Every time an AI or backend team adds an agent capability, extends an output type, or restructures a response payload, a frontend engineer has to be involved. They need to add a case, wire a component, test the binding, and ship a deploy. What should be a model-layer change becomes a cross-team sprint ticket.

At a single-agent, single-frontend scale, this is annoying. At enterprise scale—multiple agents serving multiple products across multiple frontend teams—it becomes an organizational blocker. Agent iteration speed, which is inherently fast and experimental, gets throttled to frontend release cadence. Teams start avoiding capability changes not because they're technically hard, but because the coordination overhead is too expensive.

Where It Breaks at Scale

Imagine three agents: a customer support agent, an internal analytics agent, and a sales assistant. Each is consumed by two separate Angular applications maintained by different teams. Each application has its own `ngSwitch` tree, its own component mappings, its own interpretation of what `type: 'chart'` means in terms of props and layout. When the analytics agent adds a new output format, the change has to propagate to six switch statements across two codebases owned by teams with different priorities and deployment schedules.

The system has no center of gravity. It has six points of failure.

The Right Mental Model: Agents Emit UI Specs

The architectural shift is conceptually simple: agents should not emit data that the frontend decides how to render. Agents should emit **UI specifications**—declarative descriptions of intent that a rendering layer interprets. The agent says "render a comparison table with these columns and this data." The frontend honors that intent using whatever component best fits its design system.

This separates concerns cleanly. Agents own what to show. Frontends own how to show it.

The Case for an Open Spec

This architecture only works if the specification format is shared, stable, and not owned by any single team. Proprietary formats recreate the coupling problem at a higher layer—now your agents are locked to your renderer, and your renderer is locked to your schema definitions.

What this problem space needs is an open, versioned UI specification standard: something analogous to what OpenAPI did for REST contracts. A format that agent frameworks can target, that Angular renderers can implement, and that multiple teams can build tooling around without negotiating the schema on every iteration.

That standard doesn't fully exist yet. The rest of this guide is about building toward it.

CHAPTER 2

Declarative UI Specs & the json-render Standard

Chapter 4: Declarative UI Specs & the json-render Standard

The Contract Between Agent and Interface

When a language model generates UI, you need a stable contract between what the model emits and what your frontend renders. Without one, you're parsing freeform HTML strings, maintaining brittle prompt templates, and coupling your rendering logic to whichever model happens to be in production this week. The json-render specification solves this by defining UI as structured, framework-agnostic JSON. The agent emits a document. The frontend interprets it. Neither side cares how the other is built.

Anatomy of a json-render Document

A json-render document is a tree of nodes. Each node carries three fields: `type`, `props`, and `children`.

```
{
  "type": "DataTable",
  "props": {
    "columns": ["Region", "Revenue", "YoY Growth"],
    "caption": "Q3 Performance Summary",
    "striped": true
  },
  "children": [
    {
      "type": "TableRow",
      "props": { "highlight": true },
      "children": [
        { "type": "TableCell", "props": { "value": "EMEA" } },
        { "type": "TableCell", "props": { "value": "$4.2M" } },
        { "type": "TableCell", "props": { "value": "+18%" } }
      ]
    }
  ]
}
```

`type` maps to a registered component. `props` are typed inputs. `children` are recursively resolved. The renderer never sees template syntax — just a node graph it already knows how to walk.

Why an Open Standard Matters

Framework-agnostic specs create portability. An agent prompt that targets json-render works whether the frontend is Angular, React, or Vue. Swap the renderer; keep the prompt. This matters at the enterprise level where you may have multiple teams consuming the same agent infrastructure across different stacks.

LLM prompt stability is the less obvious benefit. When your output schema is formally specified, you can provide the spec as part of the system prompt — as a JSON Schema or TypeScript interface — and get consistent, validatable output. Models respond well to schemas with explicit field descriptions and constrained `type` enumerations. Deviation rates drop significantly when the model has a schema to conform to rather than an example to imitate.

Community tooling compounds over time. Validators, mock generators, visual debuggers, and testing utilities built against the spec work across every implementation.

Conditional Rendering, Iteration, and Computed Properties

json-render handles dynamic behavior declaratively. Conditionals use a `$if` directive on the node:

```
{ "type": "AlertBanner", "props": { "message": "Quota exceeded" }, "$if":
"metrics.quotaUsed > 0.9" }
```

Iteration uses `$for` with a binding expression:

```
{ "type": "UserCard", "$for": "user in users", "props": { "name": "
{{user.name}}", "role": "{{user.role}}" } }
```

Computed properties use `{{expression}}` interpolation within prop values. Expressions are evaluated against a runtime context object supplied by the host application. The spec deliberately keeps the expression language minimal — it is not a scripting environment.

Google's A2UI Extension

Google's A2UI spec extends json-render for agent-native patterns. It introduces `intent` nodes, which describe what the agent wants to accomplish rather than prescribing a

specific component. A2UI also formalizes `action` bindings — structured event handlers that dispatch typed payloads back to the agent rather than calling arbitrary JavaScript. For teams building bidirectional agent interfaces, A2UI's interaction model is worth adopting alongside the base spec.

@cacheplane/render in Angular

`@cacheplane/render` implements json-render for Angular via the `` directive. Pass it a document and a component registry:

```
@Component({
  template: ``
})
export class AgentShellComponent {
  registry = { DataTable, TableRow, TableCell, AlertBanner, UserCard };
  agentOutput = signal(null);
}
```

The directive resolves types against the registry, instantiates components dynamically, binds props, and projects children. Streaming is handled through JSON Patch — as the agent streams tokens, partial patches are applied to the document signal incrementally. Components update as their subtree resolves. You get progressive rendering without polling.

Prompting LLMs for Spec-Compliant Output

Include the json-render JSON Schema in your system prompt. Constrain the `type` field to your registered component names. Instruct the model to emit only valid nodes and avoid HTML interpolation. Few-shot examples accelerate conformance, but schema enforcement does more work. Validate output at the boundary with a schema validator before passing documents to `` — invalid nodes should be caught there, not inside your component tree.

The spec is the API. Treat it like one.

CHAPTER 3

The Component Registry

Chapter 4: The Component Registry

The component registry is the load-bearing joint between a json-render document and your Angular application. A render spec can name any component it wants; the registry decides what that name actually means at runtime. Get this layer right and you have a stable, evolvable surface for generative UI. Get it wrong and you have a fragile string-matching problem scattered across your codebase.

Mapping Names to Components

`defineAngularRegistry()` accepts a plain object where each key is the component name a spec will reference, and each value is the corresponding Angular component class. Nothing more.

```
import { defineAngularRegistry } from '@cacheplane/render';
import { MetricCardComponent } from './metric-card.component';
import { DataTableComponent } from './data-table.component';
import { AlertBannerComponent } from './alert-banner.component';
import { ChartComponent } from './chart.component';
export const appRegistry = defineAngularRegistry({
  MetricCard: MetricCardComponent,
  DataTable: DataTableComponent,
  AlertBanner: AlertBannerComponent,
  Chart: ChartComponent,
});
```

The registry object is typed: keys are strings, values must be Angular component classes. If a component class doesn't match what the type system expects, you'll know at compile time rather than at render time.

Providing the Registry

Register the registry in your application's dependency injection tree using `provideRenderRegistry()`. This belongs in `app.config.ts` alongside your other application-level providers:

```
providers: [
  provideRenderRegistry(appRegistry),
]
```

From this point on, the render infrastructure can resolve component names anywhere in the component tree without you wiring anything else manually.

How `` Resolves Components

The `` directive is where resolution actually happens. When it receives a spec node, it reads the `component` field, looks up that string in the injected registry, and dynamically instantiates the matching Angular component into its view container. This is a straightforward `ViewContainerRef.createComponent()` call under the hood — no magic, no reflection, just Angular's native dynamic component API.

If the component tree is nested, `` recurses, resolving each child node the same way. The registry lookup happens at render time, not at compile time, which is what makes generative output possible.

Input Mapping

Once a component is resolved, `` maps the spec's `props` object onto Angular `@Input()` bindings. Each key in `props` that matches a declared `@Input()` on the component class gets assigned directly. Keys that don't match a declared input are silently ignored — the renderer does not throw on extra props, because spec generators frequently include more context than any single component needs.

Type coercion is not performed. If your `@Input()` expects a number and the spec delivers a string, that's your contract to enforce, either through input transforms (available in Angular 17+) or a validation layer upstream of the renderer.

Handling Unknown Components

When `` encounters a component name not present in the registry, the default behavior is to render nothing and emit a warning to the console. For production applications, that's usually insufficient. The registry accepts a `fallback` component that receives the original spec node as an input, letting you render a visible placeholder, log to your observability pipeline, or surface a graceful error state to the user. Treat the fallback as an error boundary: scope it, style it, and make failures observable.

Registry Versioning

Specs generated today may reference component names that your registry won't recognize in six months — or vice versa. Version your registry explicitly by maintaining

additive-only additions to registered component names and deprecating rather than removing entries. If a component's contract changes significantly, register it under a new name and keep the old binding pointing to a compatibility wrapper. This mirrors how you'd version a REST API: old consumers keep working while new specs can adopt the updated surface.

The registry is your schema. Treat it with the same discipline you'd apply to any other public interface.

CHAPTER 4

Streaming JSON Patches

Chapter 7: Streaming JSON Patches

Waiting for a complete UI specification before rendering anything is the wrong default. If your agent is generating a 50-row data table, your users should see the first rows within milliseconds of the agent starting output — not after a three-second round trip to receive, parse, and hydrate the full JSON payload. Streaming is what separates a generative UI from a slow API call with extra steps.

Why Full-Document Streaming Falls Apart

The naive approach is to re-emit the entire UI spec on each token or checkpoint. This breaks down fast. A moderately complex spec — tables, nested cards, conditional sections — can run to tens of kilobytes. Replacing the full document on every update means the renderer diffs the entire tree on every tick, Angular's change detection fires against a new object reference each time, and you're pushing O(spec size) work through the pipe regardless of how small the actual change was. At scale, this becomes a throughput problem that no amount of `OnPush` optimization will fully compensate for.

JSON Patch: The Right Primitive

RFC 6902 defines JSON Patch as a sequence of operations — `add`, `replace`, `remove`, `move`, `copy`, `test` — applied to a target JSON document using JSON Pointer paths. Rather than transmitting the document, the agent emits only what changed:

```
[
  { "op": "add", "path": "/rows/49", "value": { "id": 50, "name": "..." }
},
  { "op": "replace", "path": "/status", "value": "complete" }
]
```

This is the correct abstraction for generative UI streaming. The agent's output token stream maps cleanly to patch operations: a new row is an `add`, a corrected value is a `replace`, a retracted field is a `remove`. The frontend accumulates these deltas against a baseline spec document rather than swapping documents wholesale.

Partial-JSON Parsing and Skeleton States

Before a complete patch operation has arrived, you need somewhere to be. `@cacheplane/render` handles this through two mechanisms working in parallel.

First, partial-JSON parsing allows the renderer to extract and apply structurally valid operations from an incomplete stream. If three operations are fully formed and the fourth is mid-token, the first three are applied immediately. The incomplete operation is buffered until its closing delimiter arrives.

Second, skeleton states are declared in the baseline spec. Before any patches arrive, `@cacheplane/render` renders placeholder elements — shimmer rows, empty card outlines, header shells — based on the spec's `skeleton` annotations. As patches land, real content replaces placeholders in place. Users see a structured, stable layout from the first render frame.

Consuming Patch Events in Angular

```
@Component({ changeDetection: ChangeDetectionStrategy.OnPush })
export class ReportViewComponent {
  private render = inject(CacheplaneRenderService);
spec = signal(BASELINE_SPEC);


ngOnInit() {
    this.render
      .streamPatches('/api/agent/report')
      .pipe(takeUntilDestroyed())
      .subscribe(patch => {
        this.spec.update(current => applyPatch(current,
patch).newDocument);
      });
  }
}
```

`streamPatches` returns an `Observable` backed by an `EventSource`. Each emission is a discrete, valid patch batch. `applyPatch` from `fast-json-patch` is synchronous and returns the mutated document. Because `spec` is a signal, Angular schedules a targeted re-render without a full zone traversal.

Performance Characteristics

Patch-based updates are O(change), not O(spec size). A 50-row table receiving its fifty-first row processes exactly one `add` operation. The rest of the spec is untouched in

memory and the renderer skips it entirely. This holds regardless of how large the accumulated spec grows — the cost of each update is proportional to the diff, not the document. For enterprise UIs where specs grow complex over a session, this is not a micro-optimization; it's the architecture that keeps rendering predictable under load.

CHAPTER 5

State Management & Computed Functions

Chapter 4: State Management & Computed Functions

Static specs describe structure. They fall short the moment your UI needs to reflect derived state — a total calculated from line items, a label that changes based on a flag, a list of cards rendered from an array. For generative UI to be production-ready, the spec itself must support dynamic behavior. `@cacheplane/render` addresses this through `signalStateStore()`, computed properties, and repeat loops — mechanisms that let an agent define reactive UI logic declaratively, without pushing that responsibility into custom component code.

signalStateStore(): Agent-Managed Reactive State

`signalStateStore()` creates a Signal-backed state container that both the rendering engine and your Angular components can read from and write to. When an agent generates a spec, it can seed this store with initial values. Components rendered from that spec subscribe to the store automatically — no `@Input()` wiring, no manual change detection.

```
const store = signalStateStore({
  quantity: 3,
  unitPrice: 49.99,
  discountRate: 0.1,
  label: 'computed:subtotal * (1 - discountRate)',
  subtotal: 'computed:quantity * unitPrice',
  items: [
    { id: 1, name: 'Widget A' },
    { id: 2, name: 'Widget B' },
  ],
});
```

The store holds both raw values and computed expressions as first-class fields. The rendering engine resolves computed fields at render time and keeps them reactive — when `quantity` updates, anything that depends on `subtotal` or `label` updates automatically through Angular's Signal graph.

Computed Properties: Derived State in the Spec

Computed expressions are string-prefixed with `computed:` and evaluated against the current store context. The engine parses these at spec load time and registers them as Angular `computed()` signals derived from their dependencies.

This means an agent can express UI logic like conditional labels, formatted totals, or visibility rules entirely within the spec payload. No custom pipe. No component method. The expression evaluates deterministically given the same store state, which matters both for correctness and for testing.

The expression language is intentionally constrained — arithmetic, string interpolation, ternary operators, and property access. Complex branching belongs in components, not in spec strings.

Repeat Loops: Rendering Collections

When a spec field is an array, the `repeat` directive instructs the renderer to iterate and instantiate a child component template for each element:

```
{
  "type": "repeat",
  "source": "items",
  "template": { "type": "card", "title": "{{item.name}}" }
}
```

The renderer binds each iteration's context to the current element, making `item` available as a local reference within the template. This covers the majority of list-rendering patterns without requiring a custom Angular component to manage the loop.

Drawing the Line Between Spec Logic and Component Logic

The boundary is load-bearing. Computed expressions in the spec are appropriate for derivations that the agent needs to express — formatting, simple math, conditional values driven by store state. They are not appropriate for side effects, async operations, service calls, or branching logic with more than two outcomes.

If a computed expression starts to read like a function body, that logic belongs in a component. The spec should describe *what* the UI reflects, not *how* it gets there.

Testing Computed Functions

Because computed expressions evaluate against a plain store object, they are fully testable without rendering:

```
const store = signalStateStore({ quantity: 4, unitPrice: 25 });
expect(resolveComputed('quantity * unitPrice', store)).toBe(100);
```

`resolveComputed()` is a pure utility export — no DOM, no `TestBed`, no component fixture. This makes computed logic fast to validate in unit tests and straightforward to include in CI pipelines that verify agent-generated specs before they reach the renderer.

Deterministic evaluation is a feature, not a side effect. It means you can test agent output like any other pure function.